Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра алгоритмических языков

Протасов Александр Васильевич

# Методы аугментации данных на примере задачи классификации текстов на русском языке

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

**Научный руководитель:**

к.ф-м.н., доцент

Волкова И.А.

Москва, 2025

# Аннотация

В данной работе рассматривается влияние аугментации данных на качество моделей обработки естественного языка. Реализована библиотека с методами аугментации текстовых данных для русского языка и проведено исследование их эффективности в задаче классификации текстов.

# Содержание

**1 Введение** **4**

**2 Постановка задачи** **6**

**3 Обзор существующих работ** **7**

**4 Теоретическая часть** **15**

4.1 Языковая модель BERT 15

4.2 Реализованные методы аугментации текстов 17

4.2.1 Аугментация методом замены слов на семантически близкие 18

4.2.2 Контекстная аугментация с использованием масок и трансформеров 20

4.2.3 Аугментация методом обратного перевода 23

4.2.4 Простые эвристические методы аугментации текста 24

**5 Практическая часть** **26**

5.1 Анализ набора данных 26

5.2 Выбор классификатора и метрики 27

5.3 Описание программной реализации 29

5.4 Эксперименты 30

5.5 Анализ ошибок классификатора и их интерпретация с использованием LLM 42

**6 Заключение** **47**

**Список литературы** **48**

# 1 Введение

В современном мире, где обработка естественного языка играет ключевую роль во многих технологических и исследовательских областях, возникает необходимость в больших объемах данных. Сбор этих данных часто оказывается затруднительным из-за высоких затрат, времени и необходимости привлечения квалифицированных экспертов. В этом контексте аугментация данных выступает как важное решение.

Аугментация данных — это процесс создания новых данных путем модификации существующих. Этот подход особенно ценен при работе с ограниченными объемами данных, поскольку может значительно улучшить способность модели к обобщению и предотвращению переобучения. В машинном обучении для улучшения качества и разнообразия текстовых данных применяются различные методы аугментации, подходящие для конкретных задач. Эти методы могут включать в себя изменение структуры текста, манипуляции с его составляющими элементами, а также применение современных алгоритмов, способных генерировать новые данные на основе существующих.

Актуальность исследования методов аугментации текстовых данных в настоящее время обусловлена растущей потребностью в повышении эффективности и точности моделей обработки естественного языка. Генерация данных уже нашла широкое применение в задачах глубокого обучения, включая обработку изображений и звуковых данных. Расширение этих методов на текстовые данные открывает новые возможности для улучшения моделей машинного обучения, особенно в контексте задач, где доступные данные ограничены.

Например, задачи распознавания именованных сущностей (NER) требуют точной классификации и идентификации имен людей, организаций, местоположений и других уникальных элементов. Эффективное решение этих задач включает в себя использование методов аугментации данных, таких как замена имен и терминов, а также расшифровка аббревиатур. Такие подходы помогают модели лучше обобщать и избегать ошибок, связанных с конкретными именами и терминами.

В контексте задач классификации текстов используются уже другие методы аугментации. Классификаторы должны правильно определить, к какому классу относится данный текстовый фрагмент, будь то эмоциональная окраска, тематика или другие критерии. Методы аугментации, такие как синонимическая замена слов, изменение

порядка слов в предложениях и добавление шума в виде несущественных слов или фраз, могут значительно улучшить способность модели правильно классифицировать тексты, особенно в условиях, когда тренировочные данные ограничены или имеют неравномерное распределение классов. Такие методы способствуют увеличению объема и разнообразия тренировочного набора данных, помогая модели лучше улавливать и обобщать ключевые шаблоны, присущие каждому классу, и тем самым повышая общую точность и надежность классификации.

Кроме того, аугментация текстовых данных может повысить устойчивость моделей к потенциальным атакам, таким как намеренное внесение искажений или манипуляций с текстами. Благодаря тренировке на разнообразных модифицированных примерах, модели становятся менее восприимчивыми к попыткам манипуляций, что укрепляет их общую надежность и безопасность при эксплуатации в реальных условиях.

Расширение текстовых данных представляет собой универсальный инструмент, который может применяться в широком спектре задач машинного обучения, включая не только распознавание именованных сущностей и классификацию текстов, но и другие задачи обработки естественного языка.

В рамках данной работы основное внимание уделяется разработке и анализу методов аугментации текстов для русскоязычных наборов данных. Уникальные лингвистические особенности русского языка, такие как богатая морфология и свободный порядок слов, требуют адаптации и модификации существующих подходов к аугментации данных, что является ключевой задачей исследования. Эффективность этих методов будет оцениваться применительно к задаче классификации текстов, что позволяет не только проверить их воздействие на точность и устойчивость моделей обработки естественного языка, но и изучить способность алгоритмов обобщения на основе расширенного и модифицированного набора данных.

# 2 Постановка задачи

Задача аугментации текстов заключается в разработке алгоритмов, способных модифицировать исходный текстовый набор данных для улучшения производительности и обобщающей способности нейронных сетей.

Математическая постановка задачи определяется следующим образом.

Пусть $T$ – исходный текст, который представляет собой последовательность слов $T = \{w_1, w_2, \ldots, w_n\}$, где $w_i$ – $i$-е слово в тексте.

Пусть есть набор функций методов аугментации $A = \{A_1, A_2, \ldots, A_m\}$, где каждая функция $A_j$ представляет уникальный метод аугментации.

Пусть $S = \{s_1, s_2, \ldots, s_k\}$ – последовательность методов аугментаций, где каждый элемент $s_i$ представляет собой индекс аугментации из набора $A$. Тогда аугментированный текст $T'$ получается последовательным применением аугментаций из $S$ к исходному тексту: $T' = A_{s_k}(\ldots(A_{s_2}(A_{s_1}(T)))\ldots)$.

Задача аугментации текстов состоит в определении отображения: $T->T'$, которое задает для каждого исходного текста $T$ его аугментированное представление $T'$.

Поставленная задача подразумевает решение следующих подзадач:

1. Разработать библиотеку методов аугментации текстов для работы с русскоязычными наборами данных.

2. Выбрать подходящий русскоязычный набор текстовых данных и задачу классификации для демонстрации применимости реализованных методов аугментации.

3. Провести эксперименты и оценить качество полученных результатов.

# 3 Обзор существующих работ

В данной главе рассматривается спектр исследований, посвященных теме аугментации текстовых данных, которая занимает ключевое место в области обработки естественного языка. Работы в этой области включают в себя разработку новых методов аугментации, а также анализ и сравнение уже существующих подходов.

Исследование авторов работы [1] направлено на изучение эффективности различных техник аугментации текстовых данных в улучшении работы алгоритмов классификации для определения точки зрения и обнаружения фейковых новостей. Проведено обширное исследование влияния различных техник аугментации текстовых данных на работу распространенных алгоритмов классификации. Представлен новый подход к ансамблевому обучению на основе аугментации, который использует текстовую аугментацию для увеличения разнообразия и точности базовых обучающих моделей, улучшая таким образом общую предсказательную способность ансамбля. Исследование опирается на теорию "Мудрости толпы" в ансамблевом обучении, сосредотачиваясь на разнообразии и компетентности базовых учащихся для лучшего принятия решений.

На рис. 1 из работы [1] используется комбинация трех различных техник аугментации данных: подстановки TF-IDF, обратного перевода и вставки TF-IDF, каждая из которых сочетается с различными алгоритмами машинного обучения, включая bagged random forest и обычного RF, прежде чем результаты агрегируются с помощью GradBoost. На рис. 2 представлен другой подход ансамблевого обучения на основе аугментации: каждый классификатор обучается на разных аугментированных данных, что должно повысить разнообразие и, следовательно, общую производительность ансамбля.

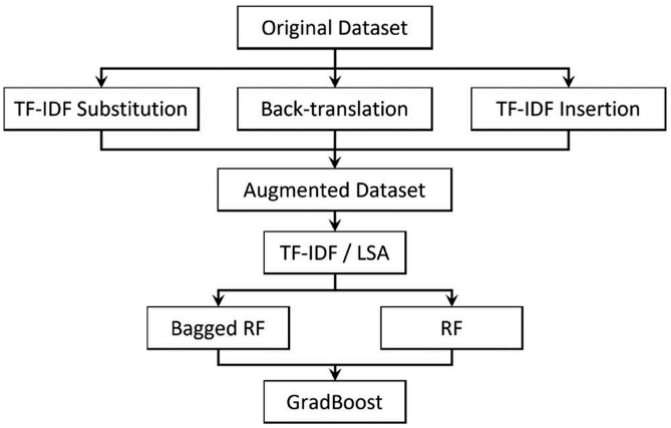


**Рис. 1:** Традиционный подход к комбинированию аугментации и стекинга [1].

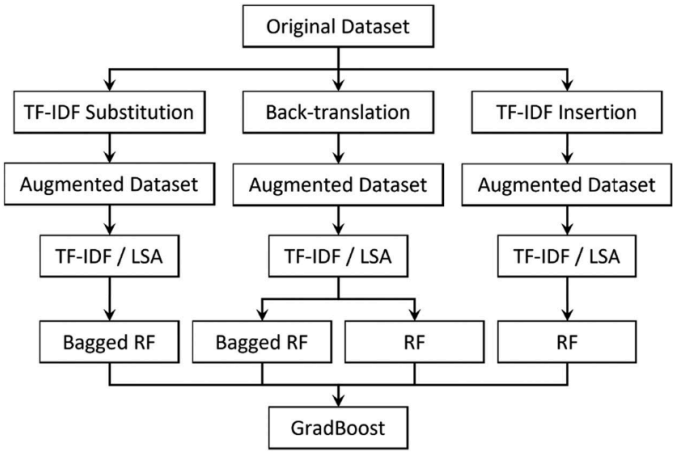


**Рис. 2**: Новый подход к ансамблевому обучению на основе аугментации данных [1].

Подход к ансамблевому обучению на основе аугментации, сочетающий стекинг и бэггинг, превзошел традиционные модели в большинстве сценариев. Этот подход привел к значительному улучшению F1-оценки и точности по сравнению с другими подходами. Также не все техники аугментации текста последовательно улучшали производительность. Например, техники, такие как BERT, показали стабильное улучшение производительности почти для всех классификаторов, в то время как другие, такие как замена синонимов, значительно варьировались в зависимости от классификатора.

В работе [2] авторы представляют собой новый подход, названный ChatAug, который эффективен для преодоления проблемы ограниченного размера выборки в задачах NLP, что является общей проблемой в сценариях обучения с небольшим объемом данных, где данные в целевой области часто недостаточны и имеют низкое качество.

Основная инновация ChatAug заключается в использовании ChatGPT, крупной языковой модели, известной своим всеобъемлющим лингвистическим обучением и способностью понимать и генерировать язык, похожий на человеческий. ChatAug использует ChatGPT для переформулировки предложений в обучающих выборках в несколько концептуально похожих, но семантически различных вариантов. Этот процесс не только увеличивает разнообразие и размер обучающего набора данных, но также поддерживает правильность и достоверность меток данных.

Другая языковая модель – BERT, известная своей способностью эффективно обрабатывать и классифицировать текст, в статье [2] применяется для оценки качества

сгенерированных данных и для тренировки модели классификации предложений на основе как исходных, так и дополненных образцов данных.

Экспериментальные результаты показали, что подход ChatAug значительно превосходит другие современные методы увеличения объема текстовых данных. Исследования проводились на наборах медицинской литературы и текстовых описаниях симптомов заболеваний. Тем не менее, авторы также признают определенные ограничения ChatAug. Например, в задачах, таких как распознавание и увеличение медицинских текстов, модель может генерировать неверные данные из-за отсутствия специализированных знаний в этой области. Чтобы решить эту проблему, авторы предлагают возможность дополнительной настройки оригинальной модели перед проведением увеличения объема данных для повышения точности в специфических областях применения.

В рамках исследования авторы работы [3] представили метод генерации текста, направленный на усовершенствование аугментации данных путем введения совместимости с предобученными моделями. Подход выделяется своей двойной стратегией, эффективно работающей как с длинными, так и с короткими текстами, используя контекстно-зависимые и контекстно-независимые процессы для генерации текстов. Такая адаптивность особенно важна, учитывая произвольный характер длины текста, причем авторы принимают границу в 280 символов — аналогичную длине твита — для различия между длинными и короткими текстами.

В работе проведена оценка реализованных методов аугментации данных в трех различных прикладных областях: анализе эмоциональной окраски, классификации новостей и кризисной информатике. Оценка метода на датасете SST-2 в условиях недостатка данных, классификация новостных статей с учетом динамично меняющихся тем и источников, а также работа в области кризисной информатики с классификацией контента социальных медиа показали значительные улучшения по сравнению с базовыми методами аугментации (EDA).

Алгоритмы (рис. 3), описанные в статье, представляют собой два метода аугментации: для коротких и длинных текстов. Отличие между ними заключается в интеграции контекста: для коротких текстов используется индексация экземпляра, тогда как для длинных текстов применяется извлечение и использование контекста из исходного текста для более целенаправленной и разнообразной генерации данных. Это позволяет сгенерированным текстам быть не только уникальными, но и тесно связанными с

исходным контентом, улучшая аугментированные данные контекстом, что особенно важно для длинных текстов.

**Algorithm 1:** Augmentation of Short Texts

**Input:** Language Model $LM$, Class data $X_c$,
Document Embedding Model $E$,
Number of instances per training data $n$

1. For each instance in $X_c$: Attach '$< |startoftext| > |i|$' to the beginning and '$< |endoftext| >$' to the end of the $i$th instance to obtain $X_{cprep}$
2. Finetune $LM$ on $X_{cprep}$ to obtain $LM_{cprep}$
3. For each k in $|X_{cprep}|$: Generate $n$ new instances with $LM_{cprep}$ and '$< |startoftext| > |i|$' as prefix to obtain $X_{gen}$
4. Embed all instances in $X_{gen}$ and $X_c$ with $E$
5. Obtain $X_{filtered}$ by including all instances of $X_{gen}$ at which the embedding representations are close to the centroid of the embedded $X_c$

**Result:** $X_{filtered}$

**Algorithm 2:** Augmentation of Long Texts

**Input:** Language Model $LM$, Class data $X_c$,
Document Embedding Model $E$,
Number of instances per training data $n$,
Function extracting context part $cont()$

1. For each instance in $X_c$: Attach '$< |startoftext| >$' to the beginning and '$< |endoftext| >$' to the end of the instances to obtain $X_{cprep}$
2. Finetune $LM$ on $X_{cprep}$ to obtain $LM_{cprep}$
3. For each $t$ in $X_{cprep}$: Generate $n$ new instances with $LM_{cprep}$ and '$< |startoftext| >$' $+ cont(t)$ as prefix to obtain $X_{gen}$
4. Embed all instances in $X_{gen}$ and $X_c$ with $E$
5. Obtain $X_{filtered}$ by including all instances of $X_{gen}$ at which the embedding representations are close to the centroid of the embedded $X_c$

**Result:** $X_{filtered}$

**Рис. 3**: Алгоритмы аугментации для коротких и длинных текстов из работы [3].

Авторы работы подводят итоги, указывая на то, что модель GPT-2, несмотря на значительное время выполнения, предоставляет более быструю альтернативу ручной разметке данных, выделяя ее практичность. Также ограниченность модели для использования преимущественно на английском языке подталкивает к необходимости применения других языковых моделей для мультиязычных задач. Результаты работы показывают, что четыре из пяти рассмотренных задач демонстрируют улучшение после добавления новых сгенерированных и отфильтрованных примеров, что приводит к увеличению среднего и максимального значений F1 на 4,8% и 4,5% соответственно. Тем не менее, было выявлено, что метод аугментации данных не приносит значительного улучшения в классификации, когда классификатор уже показывает высокую эффективность без использования дополнительных данных. Это подчеркивает, что результативность аугментации данных может варьироваться в зависимости от исходной производительности модели и специфики задачи.

В статье [4] рассматриваются техники Easy Data Augmentation (EDA), адаптированные для повышения эффективности моделей обработки естественного языка в задачах классификации текстов. Учитывая высокую вычислительную стоимость использования сложных двунаправленных моделей для аугментации данных, разработчики EDA предложили более простые, но эффективные методы, которые особенно полезны в сценариях с ограниченными наборами данных.

EDA включает четыре основные операции, предназначенные для внесения текстового разнообразия:

1. **Замена синонимами**: Этот метод сосредоточен на парафразировании через синонимию. Он включает в себя случайный выбор определенного количества слов — за исключением стоп-слов — и их замену синонимами. Эта замена использует семантическое сходство для сохранения общего смысла, при этом изменяя лексический состав текста. Техники обнаружения синонимов часто используют семантические модели, такие как Word2Vec, fastText и GloVe, которые определяют синонимы на основе близости в обученном векторном пространстве.

2. **Случайная вставка**: Для дальнейшего разнообразия текста случайная вставка выбирает синоним случайно выбранного слова и вставляет его в случайное место в тексте. Этот процесс можно повторять несколько раз, увеличивая сложность и изменчивость текста без значительного отклонения от семантики.

3. **Случайная перестановка слов**: Эта техника меняет местами два случайно выбранных слова в тексте. Повторяемая несколько раз, перестановка может тонко изменить структуру предложения и синтаксис, предоставляя модели новые паттерны для обучения, при этом сохраняя общее содержание и значение.

4. **Случайное удаление**: Удаляя слова с заранее определенной вероятностью, этот метод уменьшает избыточность текста и побуждает модель сосредоточиться на наиболее информативных частях ввода. Эта техника проверяет способность модели поддерживать производительность, несмотря на разреженность данных, имитируя сценарии, в которых могут встречаться неполные данные.

Оценка методов EDA в исследованиях показала, что даже небольшие изменения в обучающих данных могут значительно повысить производительность моделей, особенно при ограничениях по количеству данных. Например, эксперименты, проведенные с использованием EDA, продемонстрировали, что модели, обученные только на 50% аугментированных данных, могли достичь точности, сопоставимой с теми, которые обучены на 100% оригинальных данных. Это указывает на то, что EDA может эффективно удвоить размер набора данных с точки зрения его полезности для обучения.

Следующие три статьи [5,6,7] демонстрируют различные подходы к использованию контекстных представлений в задачах обработки естественного языка.

Первая статья [5] описывает инновационный метод аугментации данных для систем нейронного машинного перевода (NMT), особенно акцентируя внимание на проблемах, связанных с языковыми парами, которые имеют низкие ресурсы. Авторы рассматривают аналогичные методы из области компьютерного зрения, такие как зеркальное отображение и случайная обрезка изображений, и адаптируют эти техники для текстовых данных, сохраняя при этом смысловую эквивалентность предложений в параллельных корпусах. Предложенный подход включает в себя использование LSTM моделей для выбора контекстов, где редко встречающиеся слова могут быть заменены таким образом, чтобы обогатить тренировочные данные и улучшить качество перевода. Эксперименты, проведенные в условиях, моделирующих ограниченные ресурсы языковых пар, показали значительное улучшение качества перевода.

В работе отмечается, что процесс аугментации данных включает выбор и замену целевых редких слов из корпуса согласно предложениям модели LSTM, а также последующую замену соответствующих переводов в целевом языке, что способствует созданию новых тренировочных пар предложений. Это не только улучшает способность системы переводить редкие слова, но и делает возможным более полное их использование в тексте, что, как было замечено, ведет к генерации более длинных переводов.

Вторая статья [6] представляет собой контекстную аугментацию, применяемую в задачах классификации текстов. Авторы используют двунаправленную языковую модель для замены слов в тексте на другие слова, связанные с исходными по смыслу. Ключевым нововведением является введение архитектуры, зависимой от меток (label-conditional architecture), что позволяет модели выбирать подходящие слова для аугментации, не нарушая совместимость с метками предложений.

Экспериментальные результаты показали, что контекстная аугментация значительно улучшает качество классификации текстов, превосходя традиционные методы аугментации, такие как использование синонимов. Метод особенно эффективен, так как позволяет генерировать разнообразные замены для каждого слова, учитывая его контекст в предложении, что способствует улучшению обобщающей способности моделей машинного обучения.

Третья статья [7] вносит вклад в разработку методов создания адверсариальных примеров для атак на модели классификации текста (адверсариальные примеры — это специально модифицированные входные данные, созданные для того, чтобы ввести в заблуждение алгоритмы машинного обучения, приводя к неверной классификации или ошибкам в выводах модели) с использованием маскированной языковой модели BERT. Основной подход, названный BAE (BERT-based Adversarial Examples), заключается в замене и вставке токенов в исходный текст путем маскирования части текста и использования BERT MLM для генерации альтернативных токенов. Этот метод позволяет создавать адверсариальные примеры, которые являются не только сложными для классификации, но и сохраняют грамматическую правильность и семантическую связность.

BAE превосходит другие методы создания адверсариальных примеров, такие как замена синонимами на уровне токенов, что часто приводит к выбору слов, не соответствующих контексту и смыслу предложения. В дополнение к замене слов, BAE также включает вставку новых токенов для усиления эффективности атаки. Эксперименты показывают, что даже небольшое количество операций замены или вставки может значительно снизить точность даже таких мощных классификаторов, как BERT, более чем на 80% на некоторых наборах данных. В работе показано, что адверсариальные примеры, созданные с помощью BAE, обладают улучшенной грамматичностью и семантической целостностью. Это стало возможным благодаря использованию языковой модели и способности BERT к точному предсказанию подходящих токенов, которые логично вписываются в контекст исходного текста.

Общие черты всех исследований включают использование контекстуализированных представлений для генерации нового текста, который сохраняет семантическую связность с исходным материалом и учитывает контекст для достижения лучших результатов в своих сферах применения. Различия между статьями заключаются в их целях и методах: первая статья сосредоточена на улучшении машинного перевода за счет аугментации данных для редких слов, вторая статья развивает методы текстовой классификации через контекстные замены слов, а третья – исследует создание адверсариальных примеров для подрыва моделей классификации текста. Данные работы подчеркивают важность адаптации и инноваций в использовании контекстных данных для повышения эффективности и надежности NLP-моделей.

В процессе анализа существующих работ по аугментации данных установлено, что активное развитие данной области наблюдается преимущественно для английского языка, в то время как исследования и практики аугментации для русского языка продвигаются менее интенсивно. Этот дисбаланс подчеркивает критическую необходимость расширения границ исследований аугментации текста на другие языки, включая русский, и указывает на актуальность темы в целом. Важно отметить, что многие исследования не уделяют должного внимания вопросу применения аугментации исключительно к тренировочным данным, а не к тестовым, что является критически важным для обеспечения корректности обучения моделей.

Эти наблюдения подчеркивают необходимость дальнейшей разработки и тестирования методов аугментации, их интеграции в существующие системы машинного обучения для улучшения понимания и обработки текстовых данных.

# 4 Теоретическая часть

В данной главе рассмотрена архитектура языковой модели BERT [8] и представлена реализация разработанных методов аугментации текстов: контекстная замена и вставка слов с использованием трансформеров, обратный перевод с использованием API Google Translate [9] и нейронной моделью, семантически близкая замена и вставка слов с использованием word2vec [10], простые эвристические методы аугментации текста.

## 4.1 Языковая модель BERT

BERT (Bidirectional Encoder Representations from Transformers) — это предварительно обученная языковая модель, активно используемая в области обработки естественного языка с 2018 года [8]. Основным преимуществом BERT является двунаправленность: модель учитывает как предшествующий, так и последующий контексты слов, эффективно выявляя семантические связи.

Основой модели BERT является кодировщик архитектуры Transformer, предложенной в работе [11]. Архитектура представляет собой нейронную сеть, полностью основанную на механизме внимания, без использования рекуррентных или сверточных слоев.

Механизм внимания позволяет модели учитывать релевантность и значимость слов в предложении друг для друга, используя три типа векторов: запросы, ключи и значения. Векторы запросов сравниваются со всеми ключами через скалярное произведение, которое затем нормируется функцией softmax для получения весов внимания. Эти веса отражают важность каждого слова по отношению к текущему контексту, после чего используются для взвешенного суммирования значений.

Для расширения возможностей механизма внимания применяется многоголовое внимание (multi-head attention), позволяющее параллельно рассматривать различные аспекты входных данных. В многоголовом внимании входные матрицы $Q$, $K$ и $V$ разбиваются на подпространства, которые обрабатываются отдельно, после чего результаты объединяются и дополнительно обрабатываются через линейный слой для формирования окончательного представления.

Каждый слой кодировщика Transformer состоит из двух основных компонентов: многоголового внимания и позиционно-зависимой полносвязной нейронной сети (feed-forward neural network). Важной особенностью кодировщика является использование

residual connections (обходных соединений) и нормализации по слоям (layer normalization), которые решают проблему исчезновения или взрыва градиентов, обеспечивая стабильность обучения глубоких моделей.

В архитектуре Transformer отсутствует явный механизм учета порядка слов, поэтому дополнительно вводится позиционное кодирование. Позиционное кодирование добавляется к векторным представлениям слов на этапе предварительной обработки и позволяет модели учитывать порядок слов в предложении.

В процессе обработки модель BERT аналогично трансформеру принимает на вход последовательность токенизированных слов, расширенную двумя специальными токенами: токеном *[CLS]*, который помещается в начало каждой последовательности и его конечное скрытое состояние используется для представления общего смысла последовательности в задачах классификации, и токеном *[SEP]*, который вставляется в конец каждого предложения для их разделения. После добавления этих токенов, последовательность проходит серию преобразований на каждом из слоев кодировщика, где с помощью механизма внимания модель способна извлекать контекстуальные связи между словами на всех уровнях обрабатываемого текста.

На выходе каждого слоя модель выдает векторы, соответствующие каждому из токенов во входной последовательности. Эти векторы предопределенного размера могут быть использованы в качестве входных данных для множества задач NLP, что отличается от работы традиционных моделей трансформера, выдающих непосредственно переведенное предложение.

Многочисленные исследования показывают, что предварительное обучение модели на разнообразных задачах с использованием обширного корпуса неразмеченных текстов и последующее дообучение на конкретные цели демонстрирует значительно лучшие результаты, чем обучение модели с нуля для решения узкоспециализированных задач. BERT следует такой же двухэтапной стратегии обучения:

**Предварительное обучение (pre-training)**: Модель обучается на больших наборах неразмеченных данных, решая две основные задачи:

- Masked language modelling (MLM): случайное маскирование слов и их восстановление по контексту.

- Next sentence prediction (NSP): определение логического продолжения предложений.

Это обеспечивает модели способность понимать контекстуальные связи между словами и предложениями.

**Тонкая настройка (fine-tuning)**: после предварительного обучения модель дообучается на специализированных размеченных данных под конкретные NLP задачи. На этом этапе параметры модели настраиваются с учетом специфики задач. Для классификации текстов обычно добавляется дополнительный полносвязный слой поверх эмбеддинга токена *[CLS]*, который преобразует итоговый вектор в вероятности классов при помощи функции softmax.

На рис. 4 представлена адаптация модели BERT для классификации текстов.

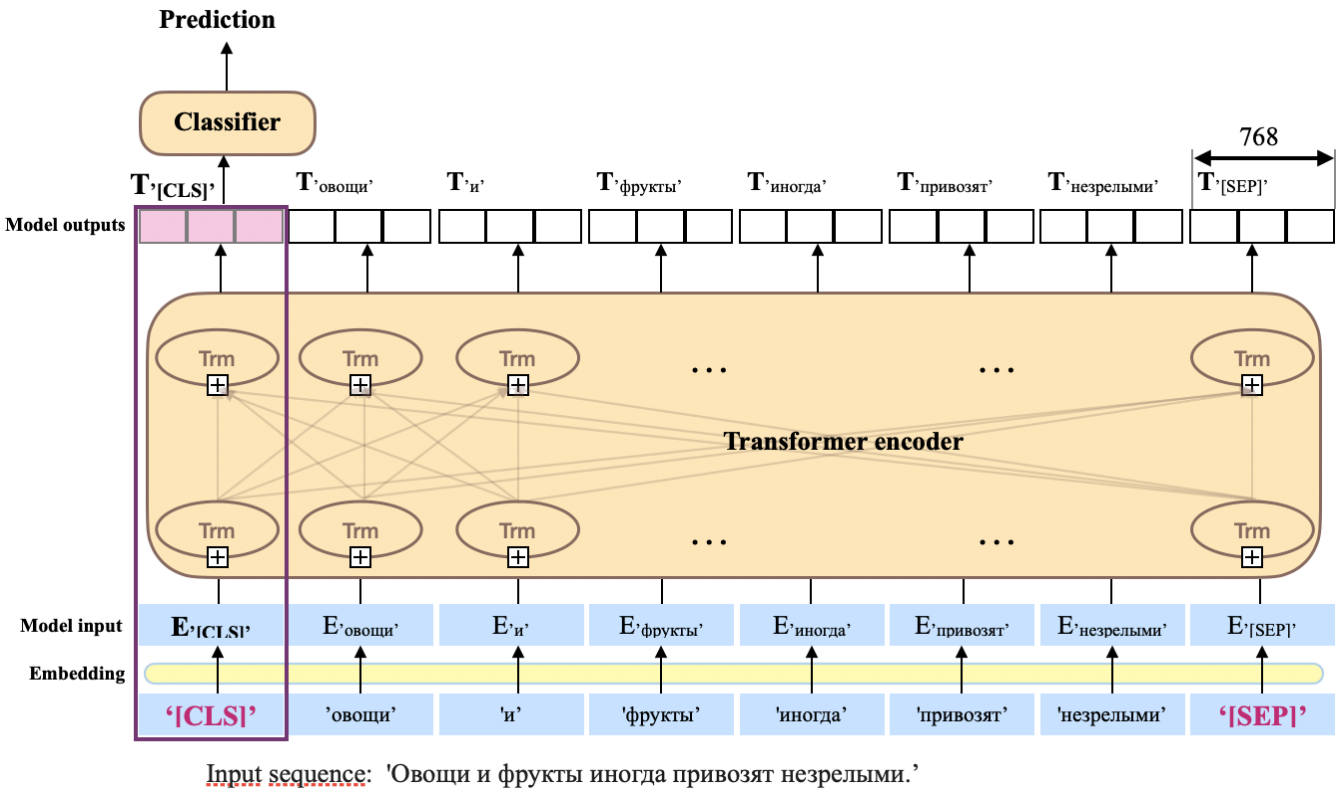


**Рис. 4**: Пример обученного BERT для классификации текстов.

## 4.2 Реализованные методы аугментации текстов

В рамках данной работы реализована библиотека с методами аугментации текстов, направленные на повышение качества классификации текстов на русском языке.

### 4.2.1 Аугментация методом замены слов на семантически близкие

Первый метод представляет собой развитие традиционного подхода замены синонимов [4], расширяя его за счет использования слов, встречающихся в схожих контекстах. Это позволяет включать в подстановку не только семантически эквивалентные термины, но также антонимы и другие контекстуально уместные слова.

Процесс замены слов семантически близкими включает следующие этапы:

- Исходный текст разбивается на отдельные слова (токены). Из полученного списка отбираются уникальные слова, за исключением стоп-слов, поскольку они редко несут важную смысловую нагрузку и их замена не приносит пользы. Из множества уникальных слов случайным образом выбирается заданная доля слов, которые будут заменены семантически близкими. Эта доля управляется параметром, задающим степень изменения текста.

Для каждого выбранного слова выполняется процедура поиска подходящего семантически близкого слова, которая включает следующие шаги:

- Определение начальной формы слова (леммы) с помощью морфологического анализатора pymorphy2 [12].

- Определение части речи. Это необходимо для корректного взаимодействия с word2vec-моделями, обученными на лемматизированных данных. Основной инструмент для этого — библиотека Natasha [13], которая возвращает тег в формате universal dependencies. При необходимости дополнительно используется анализатор mystem3 [14].

- Поиск близких по контексту слов для леммы слова с учетом его части речи. В реализованной библиотеке используется модель word2vec [10] или API RusVectores [15] с предобученной статистической моделью "ruwikiruscorpora_upos_cbow_300_10_2021" [15]. На основе полученных данных формируется список потенциальных замен, совпадающих по части речи с исходным словом (таблица 1). В случае отсутствия подходящих векторов, используется альтернативный POS-тег от mystem3 [14].

- Приведение выбранного слова для замены к грамматической форме. Из списка семантически близких слов случайным образом выбирается один, совпадающий по части речи с исходным словом. Затем морфологическая форма заменяющего слова приводится в

соответствие с грамматическими признаками исходного, такими как число, род, падеж и другие, релевантные для данной части речи.

Второй метод аугментации заключается в добавлении слов, близких по значению, в исходный текст. В отличие от первого метода, здесь исходные слова сохраняются, а объем текста увеличивается. Алгоритм работы следующий:

Исходный текст разбивается на последовательность токенов. Выполняется заданное число итераций вставки (количество задается заранее и регулирует степень изменения текста). В каждой итерации:

- На каждой итерации случайным образом выбирается слово из исходного набора токенов текста, не входящее в список стоп-слов.
- Для этого слова подбирается семантически близкое слово, аналогично методу замены: выполняется лемматизация, определение части речи, поиск синонима через аналогичные способы и приведение к нужной грамматической форме.
- Полученное слово вставляется в случайную позицию внутри текста.

| **Исходный текст** | **Без морфологической модификации** | **С морфологической модификацией** |
|---|---|---|
| упакованные продукты приезжают с плесенью | запаковать товар прибывать с плесенью | запакованные товары прибывают с плесенью |

**Таблица 1**: Аугментация с заменой слов семантически близкими до и после морфологической модификации.

Так, алгоритмы аугментации методами замены слов синонимами и вставкой синонимов описаны с помощью псевдокода 5.1, 5.2, 5.3.

**Algorithm 5.1** Алгоритм получения семантически близкого слова

**Require:** слово $w$, модель $M$, морфологический анализатор $A$
**Ensure:** синоним $s'$ с сохранением морфологии
  **function** GetSynonym$(w, M, A)$
    $n_w \leftarrow$ нормальная форма слова $w$ через $A$
    $S_w \leftarrow$ получить синонимы для $n_w$ используя модель $M$
    **if** $S_w = \emptyset$ **then return** $w$
    **end if**
    $s \leftarrow$ случайный синоним из $S_w$
    $morph_w \leftarrow$ получить морфологические характеристики слова $w$ через $A$
    $s' \leftarrow$ склонение синонима $s$ согласно морфологии $morph_w$
    **return** $s'$
  **end function**

**Algorithm 5.2** Алгоритм аугментации методом замены слов на семантически близкие

**Require:** текст $T$, доля замен $part \in [0, 1]$, модель $M$, морфологический анализатор $A$, стоп-слова $S$
**Ensure:** аугментированный текст $T'$

```
function SynonymReplacement(T, part, M, A, S)
    W ← разбить T на слова
    U ← {w ∈ W | w ∉ S}                    ▷ Уникальные слова, кроме стоп-слов
    перемешать U
    N ← ⌊|U| · part⌋                       ▷ Количество слов для замены
    R ← первые N слов из U
    for каждое w ∈ R do
        s' ← GetSynonym(w, M, A)
        if s' ≠ w then
            заменить все вхождения w в W на s'
        end if
    end for
    return объединить W в текст
end function
```

**Algorithm 5.3** Алгоритм аугментации методом вставки семантически близких слов

**Require:** текст $T$, количество вставок $n$, модель $M$, морфологический анализатор $A$
**Ensure:** аугментированный текст $T'$

```
function RandomInsertion(T, n, M, A)
    W ← разбить T на слова
    for i = 1 до n do
        w ← случайное слово из W (не стоп-слово)
        if w = None then
            continue
        end if
        s' ← GetSynonym(w, M, A)
        j ← случайный индекс в диапазоне [0, |W|]
        вставить s' в W на позицию j
    end for
    return объединить W в текст
end function
```

### 4.2.2 Контекстная аугментация с использованием масок и трансформеров

В данном подходе для аугментации текста применяется модель masked language model (MLM), способная предсказывать наиболее вероятные токены на месте "маски" с учетом окружающего контекста. Это позволяет получать более естественные и разнообразные варианты текста, чем при простых методах замены синонимами. В рамках данной работы реализованы два основных подхода: контекстная замена слов и контекстная вставка слов.

### Контекстная замена

Процесс аугментации текста на основе контекстных замен с использованием BERT включает следующие этапы:

1. Исходный текст разбивается на токены (слова), после чего формируется множество индексов токенов, не являющихся стоп-словами. На основе входного параметра, определяющего долю слов для замены, вычисляется целевое количество токенов, подлежащих аугментации, как доля от общего числа отобранных кандидатов.

2. **Итеративная замена:** Выполняется итеративный процесс до тех пор, пока не будет достигнуто заданное количество замен:

   - Случайным образом выбирается токен для замены из еще не обработанных.

   - Выбранный токен в тексте временно заменяется специальным маркером *[MASK]*.

   - Модифицированный текст с маской подается на вход предобученной MLM-модели. Модель анализирует контекст вокруг *[MASK]* и возвращает *k* наиболее вероятных слов-кандидатов для заполнения позиции *[MASK]*, а также соответствующие им оценки.

   - Из этих k кандидатов выбирается одно слово посредством взвешенного случайного выбора, где весами выступают нормализованные вероятности, предсказанные моделью.

   - Проводится проверка, не является ли выбранное слово знаком препинания. Если проверка пройдена, выбранное слово ставится на место исходного в последовательности токенов. Иначе, чтобы не нарушать структуру, на место *[MASK]* возвращается исходное слово.

3. **Формирование результата:** После завершения итераций токены объединяются обратно в текст, формируя аугментированный вариант исходного текста.

### Контекстная вставка

Второй подход — вставка новых слов на основе контекста — сохраняет все исходные слова и расширяет текст за счет добавления релевантных лексем. Алгоритм включает следующие шаги:

1. Исходный текст преобразуется в последовательность токенов. Задается параметр, определяющий, сколько слов будет добавлено в текст.

2. **Итеративная вставка:** Заданное количество раз выполняются следующие шаги:

○ Выбирается случайная позиция в текущей последовательности токенов. В эту позицию временно помещается маскирующий токен *[MASK]*, и формируется соответствующий текст.

○ Выполняется предсказание слова для замаскированной позиции, выбор одного кандидата из наиболее вероятных и проверка на допустимость вставки, как описано в алгоритме контекстной замены слов. Если слово прошло проверку, оно вставляется в последовательность токенов на выбранную случайную позицию. В противном случае, на данной итерации вставка не происходит.

3. **Формирование результата:** После выполнения всех итераций итоговая последовательность токенов объединяется в строку.

Оба метода используют единый механизм маскирования и предсказания, используя либо готовые модели Hugging Face (по умолчанию ai-forever/ruRoberta-large [16]), либо решением на базе RuLeanALBERT [17].

Так, алгоритмы аугментации с использованием контекстной вставки и замены слов описаны с помощью псевдокода 5.4, 5.5, 5.6.

**Algorithm 5.4** Предсказание слова по контексту

**Require:** маскированный текст $T_{masked}$, модель $M$, параметр $top_k$
**Ensure:** замена $replacement$
  **function** PREDICTMASKED($T_{masked}, M, top_k$)
    $variants, probs \leftarrow$ предсказания модели $M$ для маскированного токена в $T_{masked}$
    $replacement \leftarrow$ выбрать слово из $variants$ с вероятностями $probs$
    **return** $replacement$
  **end function**

**Algorithm 5.5** Алгоритм аугментации контекстной вставки слов

**Require:** текст $T$, количество вставок $n$, параметр $top_k$, модель $M$
**Ensure:** аугментированный текст $T'$
  **function** CONTEXTUALINSERTION($T, n, top_k, M$)
    $W \leftarrow$ разбить $T$ на слова
    **for** $i = 1$ до $n$ **do**
      $j \leftarrow$ случайный индекс в диапазоне $[0, |W|]$
      $W_{temp} \leftarrow$ копия $W$
      вставить $[MASK]$ в $W_{temp}$ на позицию $j$
      $T_{masked} \leftarrow$ объединить $W_{temp}$ в текст
      $replacement \leftarrow$ PREDICTMASKED($T_{masked}, M, top_k$)
      **if** $replacement$ не является пунктуацией или пробелом **then**
        вставить $replacement$ в $W$ на позицию $j$
      **end if**
    **end for**
    **return** объединить $W$ в текст
  **end function**

**Algorithm 5.6** Алгоритм аугментации контекстной замены слов

**Require:** текст $T$, доля замен $part \in [0, 1]$, параметр $top_k$, модель $M$, стоп-слова $S$
**Ensure:** аугментированный текст $T'$

```
function ContextualReplacement(T, part, top_k, M, S)
    W ← разбить T на слова
    I ← {i | W[i] ∉ S}                          ▷ Индексы не стоп-слов
    N ← ⌊|I| · part⌋                            ▷ Количество слов для замены
    W' ← W                                      ▷ Копия списка слов
    replaced ← 0                                ▷ Счетчик замененных слов
    while replaced < N и I ≠ ∅ do
        idx ← случайно выбрать элемент из I
        I ← I \ {idx}
        original ← W'[idx]
        W'[idx] ← [MASK]                        ▷ Заменить на маску
        T_masked ← объединить W' в текст
        replacement ← PredictMasked(T_masked, M, top_k)
        if replacement не является пунктуацией или пробелом then
            W'[idx] ← replacement
            replaced ← replaced + 1
        else
            W'[idx] ← original                  ▷ Вернуть оригинальное слово
        end if
    end while
    return объединить W' в текст
end function
```

### 4.2.3 Аугментация методом обратного перевода

Предполагается, что вариативность машинного перевода обеспечивает получение синтаксически и семантически близких, но лексически разнообразных версий исходного текста, что обогащает обучающий корпус.

Реализованный алгоритм обратного перевода работает следующим образом:

1. **Входные данные:** Алгоритм принимает на вход исходный текст и упорядоченную последовательность идентификаторов языков, которая определяет цепочку переводов. Также задается параметр, определяющий используемый механизм перевода.

2. **Последовательный перевод:** Исходный текст подвергается серии последовательных переводов. Процесс итеративно проходит по заданной последовательности языков. На каждой итерации текст переводится с языка, указанного текущим элементом последовательности, на язык, указанный следующим элементом. Результат перевода на одном шаге становится входными данными для следующего шага.

3. **Механизмы перевода:** В данной реализации предусмотрено два альтернативных механизма выполнения перевода на каждом шаге:

- Использование внешнего сервиса машинного перевода Google Translate [9].

- Применение локально загруженных нейросетевых моделей для машинного перевода семейства MarianMT (Helsinki-NLP [18]).

4. **Формирование результата:** После выполнения всех шагов перевода итоговый текст, полученный на последнем шаге, возвращается как результат аугментации.

Так, алгоритм аугментации методом обратного перевода описан с помощью псевдокода 5.7.

**Algorithm 5.7** Алгоритм аугментации текста методом обратного перевода

**Require:** текст $T$, список языков $L$, метод перевода $method \in \{$`"google"`, `"neural"` $\}$
**Ensure:** аугментированный текст $T'$

```
function BackTranslationAugmenter(T, L, method)
    T' ← T                                          ▷ Инициализация выходного текста
    for i ← 1 to |L| − 1 do
        src ← L[i]                                  ▷ Исходный язык
        tgt ← L[i + 1]                              ▷ Целевой язык
        if method = "google" then
            T' ← GoogleTranslate(T', src, tgt)
        else if method = "neural" then
            model, tokenizer ← загрузить модель MarianMT для пары (src, tgt)
            T' ← NeuralTranslate(T', model, tokenizer)
        end if
    end for
    return T'
end function
```

### 4.2.4 Простые эвристические методы аугментации текста

Помимо методов, использующих внешние лингвистические ресурсы или сложные модели, аугментацию текстовых данных можно осуществлять с помощью более простых стохастических операций над последовательностью слов. Такие методы, как правило, менее затратны вычислительно и вносят локальные изменения в структуру текста. В рамках данной работы реализованы два подхода: случайное удаление слов и случайная перестановка слов.

**Случайное удаление слов** реализуется путем последовательного прохода по токенам исходного текста и их независимого исключения с заданной вероятностью $p$. Оставшиеся токены объединяются в итоговую строку. Псевдокод описан в листинге 5.8.

**Algorithm 5.8** Алгоритм случайного удаления слов

**Require:** текст $T$, вероятность удаления $p \in [0, 1]$
**Ensure:** аугментированный текст $T'$
  **function** RANDOMDELETION$(T, p)$
    $W \leftarrow$ разбить $T$ на слова
    $W' \leftarrow \{w \in W \mid$ случайное число $> p\}$
    **if** $|W'| = 0$ **then**
      $W' \leftarrow \{$ случайное слово из $W\}$
    **end if**
    **return** объединить $W'$ в текст
  **end function**

**Случайная перестановка слов** заключается в выполнении заданного числа n итераций обмена местами двух случайно выбранных токенов в последовательности. После выполнения всех перестановок модифицированная последовательность преобразуется обратно в текстовую строку. Псевдокод описан в листинге 5.9.

**Algorithm 5.9** Алгоритм аугментации текста перестановкой слов

**Require:** текст $T$, количество перестановок $n$
**Ensure:** аугментированный текст $T'$
  **function** SWAPWORDS$(T, n)$
    $W \leftarrow$ разбить $T$ на слова
    **for** $i = 1$ до $n$ **do**
      $idx1, idx2 \leftarrow$ два случайных индекса из диапазона $[0, |W| - 1]$
      поменять местами $W[idx1]$ и $W[idx2]$
    **end for**
    $T' \leftarrow$ объединить $W$ в текст
    **return** $T'$
  **end function**

# 5 Практическая часть

В данной главе представлена практическая часть исследования, включающая описание используемого датасета, выбранной конфигурации классификатора BERT, применяемой метрики оценки качества модели и программной реализации. Завершается глава экспериментальной оценкой влияния аугментации текстов на качество множественной классификации пользовательских отзывов о доставке еды.

## 5.1 Анализ набора данных

Для демонстрации, анализа и верификации эффективности предложенных в рамках данной работы методов аугментации выбрана задача множественной классификации русскоязычных текстов обратной связи от пользователей [19].

Используемый набор данных подготовлен командой машинного обучения ecom.tech для сервиса доставки еды. Данный набор содержит тексты пользовательских ответов на опрос, включающий как заранее выбранные классы из предложенного списка, так и расширенные комментарии в свободной текстовой форме. Каждый текстовый экземпляр может одновременно относиться к нескольким из 50 доступных меток-классов (рис. 5). Обучающая выборка включает 4081 текста, валидационная – 447, тестовая – 3807.

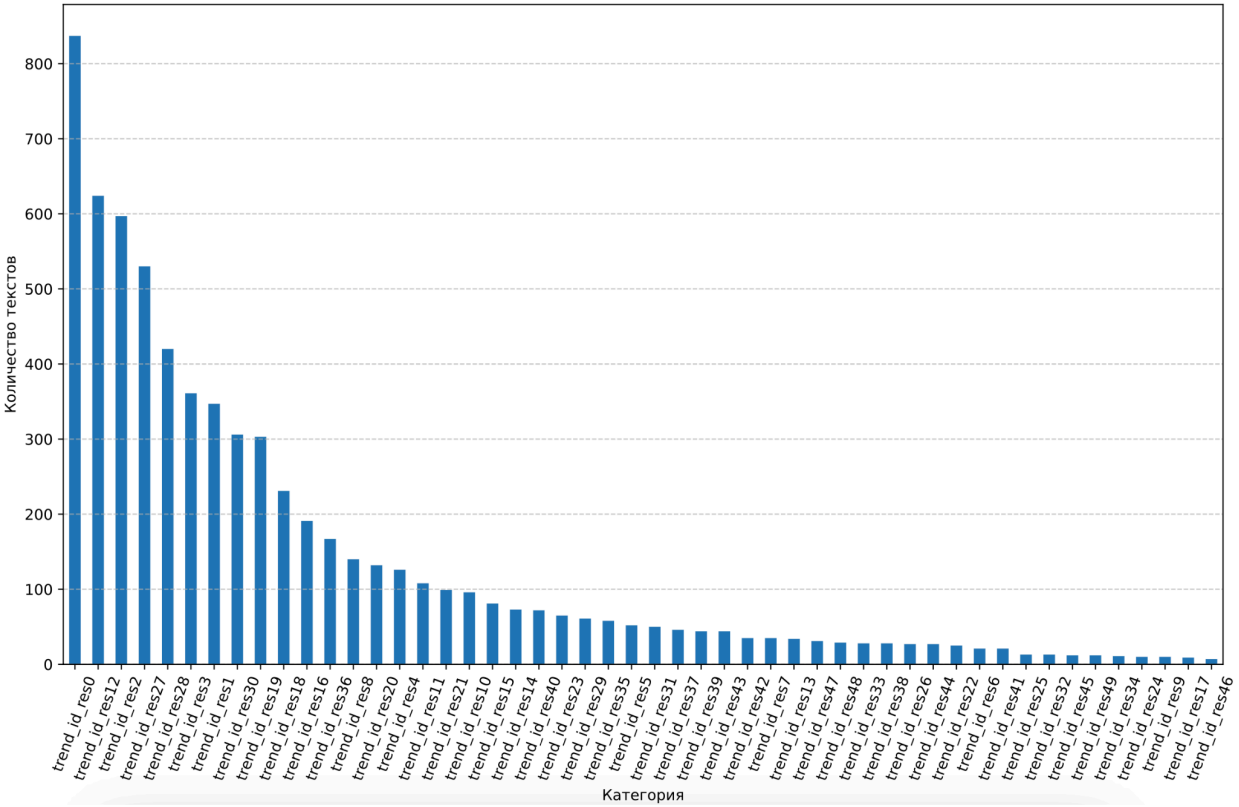


Рис. 5: Частота текстов набора данных.

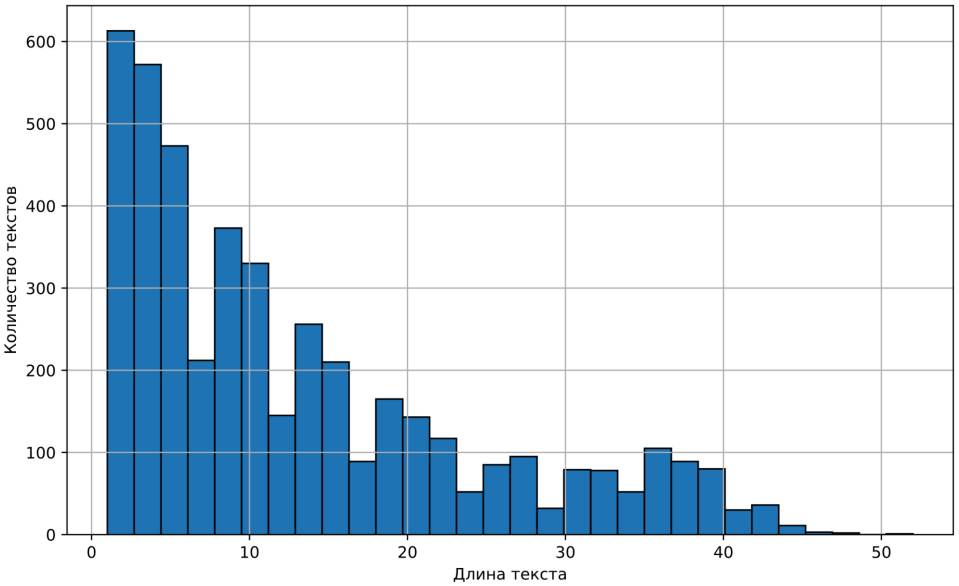

Рис. 6: Распределение длины текстов.

Каждому из 50 классов соответствует уникальный идентификатор вида trend_id_resQ, где Q $\in$ {0, 1, ..., 49}. Классы охватывают широкий спектр тем, включая как негативные аспекты обслуживания, так и позитивные отзывы. В качестве примеров можно привести: **trend_id_res2** (*Холодная еда*) — сообщение о доставке остывшего блюда; **trend_id_res19** (*Нарушение упаковки*) — жалоба на повреждение упаковки; **trend_id_res37** (*Хвалят доставщика*) — положительная оценка работы курьера.

Длины текстов варьируются от 1 до 52 слов, при среднем значении 13.16 слова. Распределение длины приведено на рис. 6, при этом тексты были предварительно очищены от пунктуации, смайликов и других неинформативных символов.

## 5.2 Выбор классификатора и метрики

В данной работе в качестве базового классификатора используется предобученная модель BERT — ai-forever/ruBert-large [16]. Модель содержит 427 миллионов параметров и использует токенизацию методом Byte Pair Encoding (BPE) с размером словаря 120 138 токенов. Обучение модели проводилось на корпусе объемом 30 ГБ, включающем тексты русской Википедии и новостных источников.

Следующим ключевым этапом после выбора модели является определение метрики для оценки качества классификации. Оптимальная метрика подбирается исходя из конкретных целей исследования и особенностей используемых данных.

В контексте задач классификации текстов, где важно учитывать баланс между точностью и полнотой прогнозов, обычно используется F-мера [20]. Значение F-меры варьируется от 0 до 1, где 0 соответствует наихудшему результату, а 1 — наилучшему. F-мера представляет собой гармоническое среднее точности ($p$) и полноты ($r$):

$$p_i = \frac{TP_i}{TP_i + FP_i}, \qquad r_i = \frac{TP_i}{TP_i + FN_i}$$

где $TP_i$ — количество текстов, правильно помеченных классом $i$;

$FP_i$ — количество текстов, неправильно помеченных классом $i$; $FN_i$ — количество текстов, которые должны были быть помечены классом $i$, но не были.

$F$-мера ($F1$) для класса $i$ затем выражается как:

$$F_i = \frac{2p_i r_i}{p_i + r_i}$$

Для получения обобщенных оценок точности и полноты по всем классам вычисляются их суммарные значения:

$$p = \frac{\sum_{i=1}^{M} TP_i}{\sum_{i=1}^{M}(TP_i + FP_i)}, \qquad r = \frac{\sum_{i=1}^{M} TP_i}{\sum_{i=1}^{M}(TP_i + FN_i)}$$

где $M$ – количество классов.

В задачах мультиклассовой классификации, таких как рассматриваемая в данной работе, для формирования целостного представления оценки эффективности классификатора используют подходы макро- и микроусреднения [20]:

$$F1_{micro} = \frac{2pr}{p + r}, \qquad F1_{macro} = \frac{\sum_{i=1}^{M} F_i}{M}$$

При макроусреднении F1-мера рассчитывается как среднее арифметическое значений F-мер по отдельным классам, при этом все классы вносят одинаковый вклад в итоговую оценку. Микроусреднение, напротив, учитывает размер классов, суммируя количество истинно положительных, ложноположительных и ложноотрицательных результатов по всем классам, после чего на основе этих суммарных значений рассчитывается общая F-мера.

Учитывая структуру набора данных и цели исследования, направленные на оценку общей способности модели правильно классифицировать тексты независимо от дисбаланса между классами, метрика F1-micro является наиболее подходящей для оценки производительности.

## 5.3 Описание программной реализации

Техническая реализация разработанных алгоритмов выполнена на языке программирования Python версии 3.10 с привлечением ряда специализированных библиотек:

- **pandas:** Используется для обработки и анализа данных, включая этапы подготовки и обработки наборов данных.
- **pymorphy2** [12]**, Natasha** [13], **mystem3** [14]**:** Морфологические анализаторы русского языка, предназначенный для приведения слов к нормальной форме и обработки их морфологических характеристик.
- **torch:** Библиотека, обеспечивающая работу с нейронными сетями и позволяющая проводить операции с тензорами в процессе обучения моделей.
- **transformers:** Предоставляет инструменты для загрузки и работы с моделями машинного обучения, в частности BertTokenizer и моделями семейства BERT.
- **gensim.models:** Используется реализация алгоритма word2vec [10] для выявления близких по значению слов на основе статистических моделей языка.

Для обучения и тестирование нейронных сетей применялась платформа Kaggle, предоставляющая доступ к мощным вычислительным ресурсам (Nvidia Tesla P100), значительно ускоряющим работу моделей глубокого обучения.

В рамках работы создан набор файлов формата .ipynb:

- **classification_texts.ipynb:** Содержит код для дообучения языковой модели BERT для решения задачи классификации текстов, включая тестирование модели и проведение экспериментов.
- **augmentation.ipynb:** Реализует предложенные методы аугментации текстов, включая замену слов на семантически близкие и использование контекстных замен.
- **dataset.ipynb:** Включает описание, анализ и процесс формирования используемого набора данных, а также создание валидационных, тестовых и обучающих выборок.

Для реализованных методов аугментации разработана библиотека **augmenta-ru** [21] на языке Python 3. Код библиотеки доступен в открытом репозитории Github [22]. На рис. 7 представлена демонстрация процесса инициализации и использования компонентов данной библиотеки.

```
[52] !pip install augmenta-ru==0.1.4

[3]  from augmenta_ru import SynonymReplacer, ContextualAugmenter, BackTranslationAugmenter, EDA
25 сек.

[4]  synonym_augmenter = SynonymReplacer(use_api=True)
     contextual_augmenter = ContextualAugmenter()
     back_translation_augmenter = BackTranslationAugmenter(method="google")
     eda_augmenter = EDA(
         synonym_replacer=synonym_augmenter,
         contextual_augmenter=contextual_augmenter
     )

[51] augmented = synonym_augmenter.augment("маленький выбор товаров хотелось бы ассортимент побольше", part=0.5)

     print(f"Аугментированный текст: {augmented}")

     Аугментированный текст: крошечный выбор продукций хотелось бы ассортимент позначительнее
```

**Рис. 7**: Пример инициализации и применения библиотеки augmenta-ru.

## 5.4 Эксперименты

В данной главе описываются эксперименты, направленные на оценку эффективности методов аугментации текстов, параметров обучения модели BERT, а также инструментов, применяемых в разработанной библиотеке с методами аугментации.

Первый эксперимент направлен на оценку производительности базовой конфигурации модели BERT, без применения алгоритмов аугментации текста. Основной задачей эксперимента является анализ метрики F1-micro на обучающей и валидационной выборках для определения степени переобучения модели.

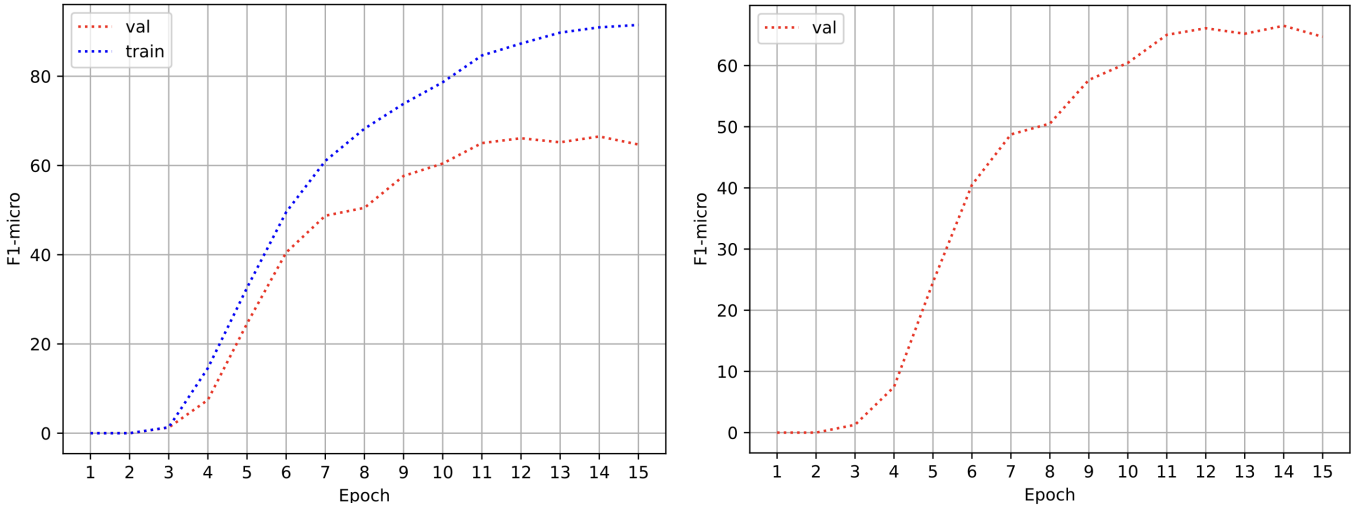


**Рис. 8**: Качество модели на обучении и валидации.

На рис. 8 слева представлены значения F1-micro на обучающем и валидационном наборах в течение 15 эпох. Видно, что после 11-ой эпохи наблюдается устойчивое расхождение между кривыми: производительность на обучении продолжает расти, в то время как на валидации достигает плато (около 65%) и начинает колебаться. Это является признаком переобучения. Правый график дублирует F1-micro на валидационной выборке отдельно, что позволяет лучше отслеживать динамику и выявить оптимальную точку остановки обучения – 11-я эпоха.

Во втором эксперименте проводится оценка влияния ключевых гиперпараметров — размера батча (batch size) и скорости обучения (learning rate) — на эффективность классификации текстов с использованием модели BERT. Указанные параметры играют важную роль в процессе обучения, так как определяют как стабильность и скорость сходимости, так и итоговое качество модели. Все эксперименты выполняются на исходном датасете без применения алгоритмов текстовой аугментации.

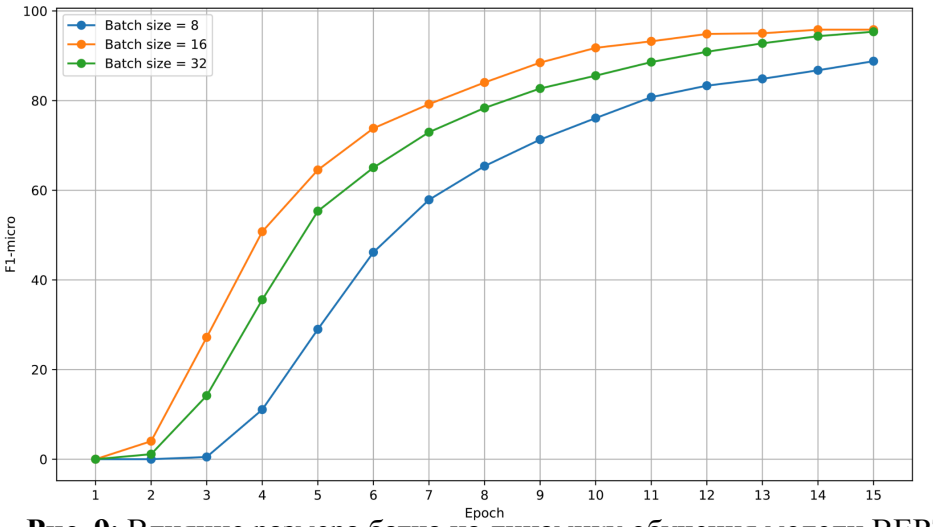


**Рис. 9**: Влияние размера батча на динамику обучения модели BERT.

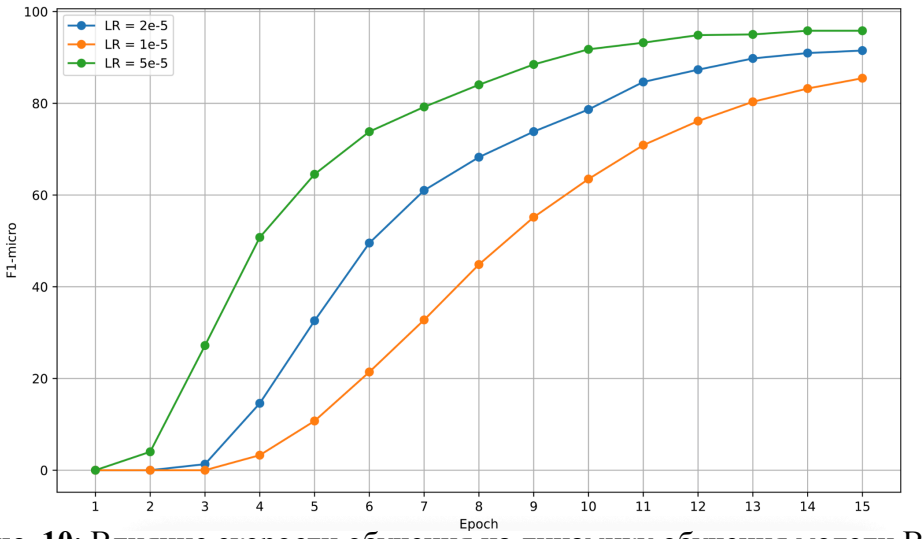


**Рис. 10**: Влияние скорости обучения на динамику обучения модели BERT.

Эксперимент показал, что размер батча существенно влияет на динамику обучения модели BERT (рис. 9). Наиболее быстрый рост F1-micro на ранних эпохах продемонстрировала конфигурация с размером батча равным 16, достигнув максимального значения 95.82% к 15-й эпохе. Это указывает на эффективный баланс между частотой обновлений весов и стабильностью градиентных шагов.

Размер батча 32 показывает немного уступающие итоговые значения по сравнению с размера батча 16, что может быть обусловлено взаимодействием размера батча с другими гиперпараметрами, в частности скоростью обучения, а также более сглаженной траекторией оптимизации, характерной для больших батчей. Размер батча равным 8 обеспечил самый медленный рост метрики и итоговое значение 88.79%, что объясняется высокой стохастичностью градиентных обновлений, что, с одной стороны, затрудняет быструю сходимость, но с другой — потенциально обладает регуляризирующим эффектом и может приводить к лучшим результатам при увеличении количества эпох.

При фиксированном размере батча (16) наилучшее качество обучения продемонстрировала конфигурация со скоростью обучения, равным 5e-5, обеспечив наиболее быстрый рост F1-micro и достижение значения 95.82% к 13-й эпохе (рис. 10). Это подтверждает, что при ограниченном числе эпох более высокие значения скорости обучения способствуют ускоренной сходимости модели. Следует учитывать, что оптимальное значение скорости обучения зависит от размера батча: увеличение размера батча обычно допускает использование более высокой скорости обучения, тогда как меньшие значения требуют ее снижения для устойчивой сходимости. В данном эксперименте снижение learning rate при batch size = 16 замедлило обучение.

Цель третьего эксперимента — оценить влияние лемматизации на работу модели BERT в задаче классификации текстов. Лемматизация представляет собой процесс приведения слов к их нормальной форме и часто используется как этап предобработки данных. Требуется проверить, оказывает ли лемматизация положительное или отрицательное влияние на способность модели к обобщению и обработке разнообразных словоформ.

| **Обработка текстовых данных** | **F1-micro на тесте** |
|---|---|
| С лемматизацией | 0.758 |
| Без лемматизации | **0.781** |

**Таблица 2**: Влияние лемматизации на работу модели BERT.

Результаты, представленные в таблице 2, показывают, что лемматизация не всегда приводит к улучшению качества классификации текстов при использовании модели BERT. В данном эксперименте предварительная обработка текста с применением лемматизации не способствовала повышению обобщающей способности модели.

Одним из подходов к повышению надежности предсказаний в задаче множественной классификации является введение порога уверенности — минимального значения вероятности, при котором модель сохраняет предсказанный класс. В данном эксперименте проводится оценка влияния порога уверенности на метрику F1-micro.

| **Порог уверенности** | **F1-micro на тесте** |
|---|---|
| 0.5 | 0.76 |
| 0.7 | **0.78** |
| 0.9 | 0.75 |

**Таблица 3**: F1-micro при различных значениях порога уверенности.

Результаты эксперимента в таблице 3 демонстрируют, что оптимальное значение порога уверенности для используемой модели находится в диапазоне 0.7. При этом значении достигается наивысший результат метрики F1-micro, что указывает на улучшение баланса между полнотой и точностью при отбрасывании наименее уверенных предсказаний.

В дополнение к описанию используемых методов аугментации, в данном разделе приведены эксперименты, обосновывающие выбор инструмента морфологической обработки текста. В качестве сравнения взяты библиотеки pymorphy2 [12], Natasha [13], и mystem3 [14]. Проводится их сравнительная оценка по двум ключевым критериям: времени обработки и точности определения части речи. Эти параметры непосредственно влияют как на производительность алгоритма аугментации, так и на его влияние на итоговое качество модели классификации.

| **Библиотека** | **Время работы функции определения части речи (мин)** |
|---|---|
| Natasha | 0.03 |
| mystem3 | 12.23 |
| pymorphy2 | **0.01** |

**Таблица 4**: Время работы функций определения части речи (2000 слов).

| **Библиотека** | **Количество нераспознанных слов** |
|---|---|
| Natasha | 151 |
| mystem3 | **119** |
| pymorphy2 | 447 |

**Таблица 5**: Количество нераспознанных слов (2000 слов).

Первый эксперимент направлен на измерение времени выполнения функции, определяющей часть речи слова. Учитывая масштаб обрабатываемых данных, данный показатель имеет критическое значение для практической применимости методов. Результаты представлены в таблице 4. Наиболее высокую скорость продемонстрировали библиотеки pymorphy2 и Natasha — 0.01 и 0.03 минуты соответственно при обработке 2000 слов. В то же время библиотека mystem3 показала существенно более длительное время выполнения — 12.23 минуты.

Второй эксперимент посвящен оценке качества морфологического анализа. Для этого сравнивается количество слов, нераспознанных предобученной моделью word2vec, обученной на корпусе, приведенном к начальным формам с указанием части речи. Поскольку корректное определение леммы и грамматической категории слова напрямую влияет на способность модели подбирать контекстно близкие векторы. Наименьшее количество нераспознанных слов зафиксировано у mystem3 — 119. Библиотека Natasha показала близкий результат (151), в то время как pymorphy2 продемонстрировала наихудшие показатели — 447 нераспознанных слов (таблица 5).

На основании полученных данных можно сделать следующие выводы: pymorphy2 является наилучшим вариантом с точки зрения скорости обработки, однако качество морфологического анализа этой библиотеки ограничено; mystem3, напротив, демонстрирует высокую точность, однако значительно уступает по скорости обработки, что ограничивает ее применимость в задачах с большим объемом данных; библиотека Natasha демонстрирует сбалансированные показатели по скорости и точности, в связи с чем выбрана в качестве основного инструмента морфологической обработки в реализованных методах аугментации.

В следующем эксперименте проводится оценка эффективности комбинированного подхода к морфологической разметке, при котором основная библиотека Natasha дополняется одной из альтернативных — pymorphy2 или mystem3. Целью эксперимента

является определение, какая из дополнительных библиотек обеспечивает более качественное покрытие в тех случаях, когда библиотека Natasha не обеспечивает корректное определение части речи.

Для оценки рассматриваются два критерия: количество нераспознанных слов (то есть слов, для которых не удалось определить часть речи) и время выполнения анализа при обработке корпуса объемом 2000 слов. Результаты приведены в таблице 6.

| **Библиотеки** | **Количество нераспознанных слов** | **Время работы (мин)** |
|---|---|---|
| Natasha и pymorphy2 | 142 | 0.41 |
| Natasha и mystem3 | **109** | 8.7 |

**Таблица 6**: Сравнение эффективности комбинации библиотек Natasha с pymorphy2 и mystem3 в определении частей речи (2000 слов).

Результаты из таблицы 6 демонстрируют, что комбинированное использование библиотеки Natasha с дополнительными средствами морфологического анализа позволяет сократить количество нераспознанных слов по сравнению с изолированным применением Natasha. Наилучшее качество показала связка Natasha и mystem3, обеспечив минимальное количество нераспознанных слов (109), однако при этом время обработки возросло до 8.7 минут. В то же время комбинация Natasha и pymorphy2 обеспечила более высокую скорость обработки (0.41 минуты) при снижении точности морфологического анализа (142 нераспознанных слова). Выбор конкретной комбинации библиотек зависит от приоритетов задачи: при ограничениях по времени обработки предпочтительнее использовать связку Natasha и pymorphy2, в то время как приоритет максимального качества может обоснованно вести к выбору связки Natasha и mystem3, несмотря на увеличенные затраты времени.

Для метода аугментации на основе семантически близкой замены слов выбрана комбинация Natasha и mystem3, показавшая наилучшее качество морфологического анализа, что критически важно для корректного выбора контекстно релевантных замен.

В следующем эксперименте проведена оценка эффективности модели BERT (DeepPavlov/rubert-base-cased [23]) в задаче контекстной замены слов (fill-mask). Целью эксперимента является исследование способности модели корректно подбирать замену для заданного слова на основе контекста.

| **Параметр** | **Количество слов** |
|---|---|
| Слова заменены пукнутацией | 106 |
| Для слов не найдено замены | 76 |
| Всего уникальных слов | 1003 |
| Общее количество слов | 2000 |

**Таблица 7**: Результаты контекстной замены слов.

Анализ данных, представленных в таблице 7, показывает, что случаи некорректной замены (на знак пунктуации) либо полного отсутствия замены составляют около 18% от общего числа уникальных слов. Такой уровень ошибок потенциально снижает качество формируемого аугментированного текста.

В рамках эксперимента рассмотрены два подхода к обработке слов, для которых модель BERT возвращала знак пунктуации или не находила подходящей замены:

- Первый подход предусматривает использование семантически близкой замены для таких слов с применением соответствующего метода аугментации.

- Второй подход заключается в отбрасывании подобных слов, при этом регулирование общего количества замен осуществлялось через входной параметр, определяющий долю слов, подлежащих замене.

При использовании первого подхода количество замен на пунктуацию сокращено на 70%. При реализации второго подхода отмечено снижение фактической доли заменяемых слов. Для ее восстановления скорректирован входной параметр, задающий долю слов для замены, что позволило сохранить целевой порог аугментации.

Результаты предыдущего эксперимента показали, что точность подбора слов при контекстной замене существенно влияет на качество аугментированных текстов. При этом корректность замен может зависеть как от специфики корпуса, на котором обучалась языковая модель, так и от архитектурных особенностей самой модели и объема ее языковых знаний.

В связи с этим проведен дополнительный эксперимент, направленный на сравнительный анализ качества работы различных русскоязычных BERT-подобных моделей в задаче контекстной замены слов. Целью эксперимента является определение

того, как различные модели справляются с задачей заполнения пропусков в тексте и какой процент некорректных и отсутствующих замен наблюдается в каждом случае.

| **Модель** | **Доля неуспешных замен (%)** |
|---|---|
| RuLeanALBERT [17] | **14%** |
| ai-forever/ruRoberta-large [18] | **12%** |
| ai-forever/ruBert-large [18] | 15% |
| ai-forever/ruBert-base [18] | 20% |
| DeepPavlov/rubert-base-cased [23] | 18% |

**Таблица 8**: Качество контекстной замены слов различными моделями.

Результаты эксперимента, представленные в таблице 8, показывают, что качество контекстной замены слов определяется не только размерами модели, но и характеристиками ее предобучения. Модели ai-forever/ruRoberta-large и RuLeanALBERT продемонстрировали наименьший процент некорректных замен, подтверждая высокую способность к пониманию контекста. Следует отметить, что эксперимент проводился на выборке текстов из набора данных, используемого в рамках текущей работы, содержащей 2000 слов, включая 1003 уникальные лексемы. Для получения более полной картины требуется дальнейший анализ эффективность моделей на текстах различных тематик и объемов.

В соответствии с полученными результатами, для реализации контекстной замены слов в разработанной библиотеке по умолчанию используются модели ruRoberta-large и RuLeanALBERT, как показавшие наилучшие результаты.

В следующем эксперименте проводится оценка влияния применения методов аугментации текстов на эффективность обучения классификатора. Анализируются четыре варианта построения обучающей выборки: без использования аугментации, с одной аугментированной версией каждого исходного текста, с двумя аугментированными версиями (путем независимого случайного выбора слов для каждой аугментации) и с тремя аугментированными версиями. Целью эксперимента является определение, каким образом увеличение числа аугментированных примеров влияет на скорость сходимости модели и итоговое качество классификации.

| **Сценарий** | **1 аугментация** | **2 аугментации** | **3 аугментации** |
|---|---|---|---|
| Без аугментации | 78% | | |
| Контекстная замена (BERT) | 78.6% | **80.3%** | 79.9% |
| Семантически близкая замена слов | 79.3% | 79.6% | 79.6% |
| Обратный перевод (API Google Translate) | **79.5%** | 79.7% | 80% |
| Обратный нейронный перевод | 78.7% | 79.8% | **80.1%** |

**Таблица 9**: Результаты F1-micro на тестовой выборке при различном количестве аугментированных версий текста.

Анализ результатов на тестовой выборке (таблица 9) показывает, что применение аугментации текстов приводит к повышению качества классификации по сравнению с обучением без аугментаций. Максимальное значение F1-micro достигнуто при использовании двух аугментированных версий текста, что указывает на положительное влияние увеличения обучающей выборки. При дальнейшем увеличении числа аугментаций до трех прирост качества либо стабилизируется, либо варьируется в зависимости от применяемого метода.

При более детальном рассмотрении эффективности отдельных методов аугментации (таблица 9) в контексте задачи множественной классификации русскоязычных отзывов о доставке еды, наилучший результат продемонстрировал метод контекстной замены на основе модели BERT при генерации двух аугментированных версий (F1-micro = 80.3%). Преимущество данного подхода может быть связано со способностью трансформерных моделей учитывать контекст предложения при формировании замен, что способствует сохранению семантической согласованности текста. Методы обратного перевода, как с использованием API Google Translate, так и с применением нейронной модели, продемонстрировали высокую эффективность, достигнув значений F1-micro 80.0% и 80.1% соответственно при использовании трех аугментаций. Их эффективность может объясняться тем, что генерируемые перефразированные варианты исходных текстов с изменением синтаксической структуры и лексического наполнения повышают робастность модели к вариативности естественного языка. Метод семантически близкой замены слов, основанный на использовании статических векторных представлений, показал меньший прирост качества, достигнув максимального значения F1-micro 79.6%. Это может быть связано с тем, что при замене слов на семантически близкие без учета

полного контекста предложения возможно нарушение исходного смысла текста за счет подстановки лексем с противоположной или некорректной смысловой окраской. Вместе с тем данный метод потенциально может способствовать большему повышению устойчивости модели к адверсариальным атакам.

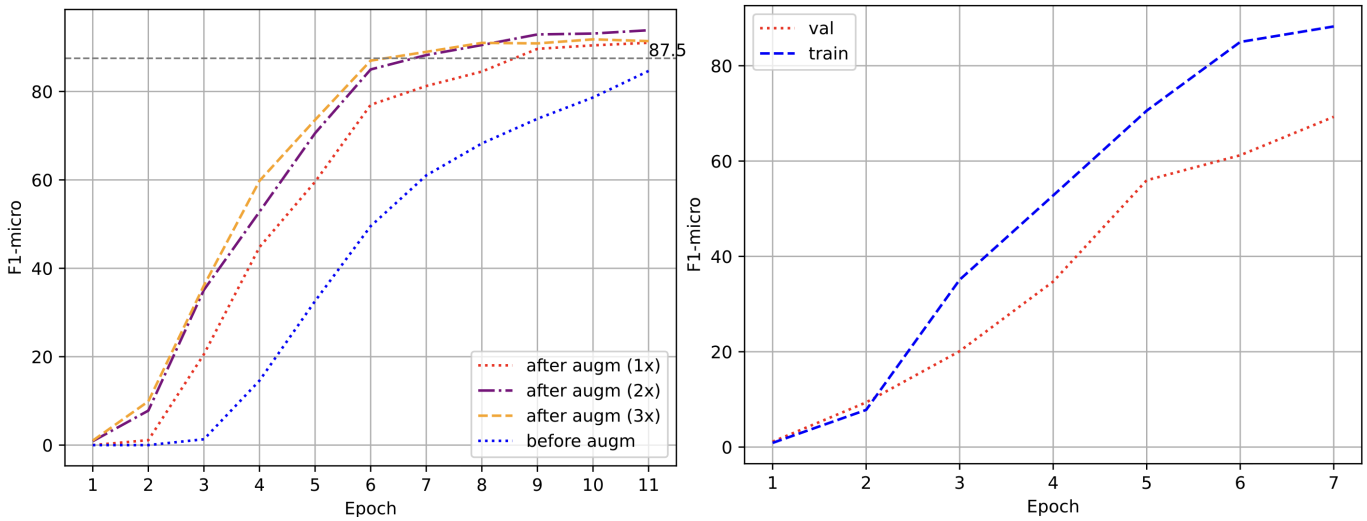


**Рис. 11**: Динамика обучения классификатора для метода контекстной замены с использованием BERT: слева — влияние количества аугментированных версий текста на процесс обучения, справа — сравнение кривых обучения и валидации для оптимального сценария с двумя аугментациями.

Учитывая, что метод контекстной замены с использованием BERT показал наилучшие результаты на тестовой выборке при двух аугментациях, дополнительно рассмотрена динамика процесса обучения классификатора при использовании данного метода с различным количеством аугментированных версий текста. Результаты представлены на рис. 11: слева показаны кривые обучения классификатора для различных сценариев аугментации. Видно, что использование одной, двух и трех аугментированных версий текста приводит к ускорению сходимости модели и достижению более высоких значений метрики F1-micro на обучающей выборке по сравнению с обучением без аугментации.

Максимальные результаты показал сценарий с двумя аугментированными версиями текста, при котором наблюдается устойчивый рост качества в процессе обучения. При этом достижение близких к максимальным значений F1-micro фиксируется уже на седьмой эпохе, что указывает на ускорение процесса обучения и позволяет завершать его на более ранних итерациях без значительной потери качества. На рис. 11 справа приведены кривые обучения и валидации для оптимального сценария с двумя аугментациями. Наблюдается стабильный рост метрик на обеих выборках, при этом сохраняющийся разрыв между ними

остается относительно постоянным на протяжении всего процесса обучения, что указывает на контролируемое обобщение модели и отсутствие выраженного переобучения.

В предыдущих экспериментах оценивалась эффективность различных методов аугментации текстов при их индивидуальном применении. Однако каждый из методов имеет свои преимущества и ограничения: например, контекстная замена слов обеспечивает высокую семантическую согласованность, тогда как обратный перевод способствует генерации перефразированных формулировок.

Для каждого текста из обучающей выборки сгенерировано две аугментированные версии, каждая из которых получена последовательным применением двух различных методов. Цель эксперимента — определить, способствует ли комбинирование методов улучшению результатов по сравнению с их индивидуальным использованием.

| **Сценарий** | **F1-micro на тесте** |
|---|---|
| Замена слов семантически близкими, затем обратный перевод | 80% |
| Контекстная замена слов, затем обратный перевод | **80.5%** |
| Замена слов семантически близкими, затем перестановка случайных позиций слов | 79% |
| Замена и вставка слов семантически близкими, затем случайное удаление слов | 78.8% |

**Таблица 10**: Результаты комбинаций реализованных методов аугментации на тестовой выборке.

Анализ результатов, представленных в таблице 10, позволяет сделать следующие выводы. Все рассмотренные комбинации методов аугментации обеспечили повышение качества классификации по сравнению с базовым сценарием без аугментации (F1-micro = 78.0%).

Наиболее высокие показатели достигнуты при последовательном применении контекстной замены слов на основе BERT и обратного перевода, где метрика F1-micro составила 80.5%. Данный результат незначительно превосходит оптимальный показатель, полученный при использовании монометода (контекстная замена на основе BERT с двумя итерациями аугментации, F1-micro = 80.3%). Такое улучшение качества можно объяснить тем, что комбинация двух наиболее эффективных методов позволяет обеспечить

разнообразие как на лексическом уровне (за счет контекстной замены на основе BERT), так и на структурно-синтаксическом уровне (за счет обратного перевода). Такое комплексное изменение исходного текста способствует повышению робастности модели к различным формулировкам.

Комбинации, включающие методы внесения шума – перестановку слов и случайное удаление – после семантической замены, показали заметно более низкие результаты (79.0% и 78.8% соответственно). Это говорит о том, что такие операции могут нарушать грамматическую структуру и семантическую связность текста в большей степени, чем это полезно для обучения модели в рамках данной задачи.

В результате генерации двух аугментированных версий для каждого текста объем обучающей выборки увеличился до 8841 примера, из которых 4760 получены посредством аугментации. При формировании итогового корпуса не учитывались мажоритарные классы, представленные в виде отдельных меток trend_id_resQ, а также бинарные комбинации вида (trend_id_res0, trend_id_resQ), где Q ∈ {1, 2, 3, 12, 27, 28, 30}. Соотношение оригинальных и аугментированных данных представлено на рис. 12.

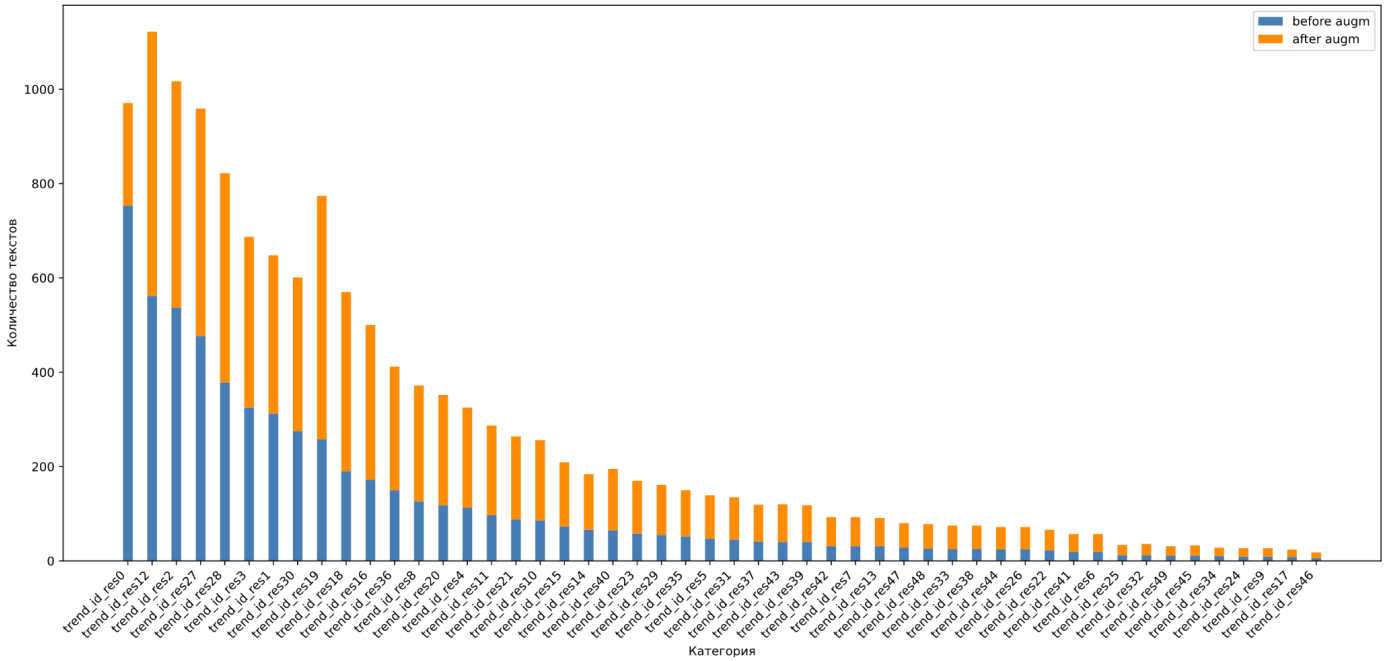


**Рис. 12**: Распределение количества аугментированных и оригинальных текстов в обучающей выборке.

В целях дополнительной верификации эффективности реализованных методов аугментации проведен эксперимент с другим набором данных, состоящим из новостных текстов. В качестве источника использована коллекция материалов с портала Tass.com,

предоставляемая проектом Natasha [13] через библиотеку Corus. Данный корпус включает 55 472 текста, распределенных по 14 тематическим категориям. В результате расширения обучающей выборки общее количество текстов увеличилось с 44 920 до 60 560. Применение аугментации с использованием метода контекстной замены на основе BERT способствовало улучшению качества модели: значение F1-micro увеличилось до 0.85 по сравнению с 0.82 при обучении без аугментации.

## 5.5 Анализ ошибок классификатора и их интерпретация с использованием LLM

Оценка эффективности нейронных моделей является ключевым этапом в рамках решения задачи множественной классификации текстов. При этом агрегированные метрики, такие как F1-micro, хотя и отражают общее качество модели, не позволяют выявить систематические ошибки и особенности поведения модели на уровне отдельных классов. Для обеспечения полной оценки проведен детальный анализ предсказаний классификатора на тестовой выборке, с фокусом на расхождения с эталонной разметкой.

На первом этапе выполнена количественная оценка ошибок, выраженных через показатели ложноположительных (FP) и ложноотрицательных (FN) предсказаний по каждому из классов. Анализ показал, что ошибки модели распределены неравномерно: основная их часть сосредоточена в ограниченном числе классов, тогда как большая часть классов демонстрирует минимальное количество ошибочных предсказаний. Для последующего анализа отобраны только те классы, которые, с одной стороны, имели не менее 100 примеров в тестовой выборке, а с другой — демонстрировали долю ошибочных предсказаний свыше 20 % (FN и FP) относительно числа тестовых примеров данного класса.

| **ID класса** | **Название класса** | **Общее число ошибок** | **Доля ошибок** |
|---|---|---|---|
| 1 | Доставка стала долгой | 172 | 27.6% |
| 3 | Регулярные опоздания | 138 | 21.3% |
| 4 | Не отследить реальное время доставки | 58 | 25.7% |
| 15 | Просроченные товары | 32 | 22.2% |
| 35 | Долго на сборке | 28 | 27% |

**Таблица 11**: Классы с наибольшей долей ошибок классификатора BERT.

Общее количество уникальных текстов, содержащих хотя бы одну ошибку классификации (FN или FP) по классам (таблица 11), отобранным согласно ранее описанным критериям, составило 392. Следует также отметить, что для значительной части классов (30 из 50) общее количество ошибок (FN и FP) не превысило 10.

В целях расширения анализа и получения дополнительной интерпретации сложных примеров в исследование включен подход с использованием современных больших языковых моделей (LLM). Благодаря способности учитывать контекст, выявлять семантические связи и интерпретировать тексты на уровне смысла, LLM рассматривается в качестве вспомогательного инструмента анализа ошибок модели. Хотя такие модели также подвержены ошибкам, их использование позволяет получить независимую точку зрения, отличную от поведения обученной архитектуры.

**LLM в роли независимого классификатора.** В рамках первого эксперимента большой языковой модели предлагается выполнить классификацию 100 текстов, отобранных из множества ошибочно классифицированных примеров тестовой выборки. На вход модели передаются тексты вместе с полным перечнем возможных классов и их описаниями. Классификация осуществляется в режиме zero-shot, то есть без предварительного обучения на данных конкретной задачи. В процессе предсказания модель опирается исключительно на представленные описания классов, общие языковые знания и информацию о предметной области, связанной с отзывами о сервисах доставки. Цель данного этапа — оценить, насколько успешно языковая модель справляется с теми примерами, по которым классификатор, обученный в рамках текущей работы, допустил ошибки, а также выяснить, являются ли эти ошибки следствием ограничений модели либо объективной сложности соответствующих текстов. В качестве LLM в эксперименте использовались RuadaptQwen 2.5 QWQ от RefalMachine [24], YandexGPT 5 Pro от Яндекс [25], Claude 3.7 от Anthropic [26] и Gemini 2.5 Pro от Google [27].

| **Название модели** | **Доля верных ответов** |
|---|---|
| RuadaptQwen2.5 QWQ | 37% |
| YandexGPT 5 Pro | 37% |
| Claude 3.7 | 40% |
| Gemini 2.5 Pro | 32% |

**Таблица 12**: Результаты zero-shot классификации 100 ошибочных примеров: доля верных ответов по моделям.

Как видно из таблицы 12, ни одна из использованных LLM не достигла высокой точности при классификации текстов, отобранных из множества ошибочно предсказанных примеров модели, обученной в рамках настоящего исследования. Доля верных предсказаний варьировалась от 32 % (Gemini 2.5 Pro) до 40 % (Claude 3.7), что свидетельствует о значительной сложности рассматриваемой подвыборки. Учитывая, что все тексты в ней изначально вызвали затруднения у дообученной модели BERT, подобный уровень ошибок среди LLM-предсказаний не является неожиданным.

Данный результат позволяет предположить, что затруднения в классификации обусловлены не только архитектурными особенностями конкретных моделей или ограничениями обучающего процесса, но и объективной сложностью самих текстов. Многие примеры содержат неявные признаки, перекрывающиеся темы и формулировки, допускающие множественную интерпретацию, что затрудняет корректную классификацию. Таким образом, ошибки классификатора представляют собой нетривиальные случаи, которые остаются трудными и для современных языковых моделей. Это подтверждает целесообразность дальнейшего анализа этих примеров с использованием LLM уже в роли интерпретатора, с целью выявления причин расхождений между предсказаниями классификатора, обученного в рамках текущей работы, и эталонной разметкой.

**LLM в роли аналитика и "судьи".** Во втором эксперименте большая языковая модель используется для качественного анализа расхождений между предсказаниями классификатора, обученного в рамках настоящего исследования, и эталонной разметкой. Каждой модели предоставлялся исходный текст отзыва, два варианта классификации (один соответствующий предсказанию классификатора, другой — эталонной разметке), а также полный перечень возможных классов с их описаниями. При этом не указывалось, какой из вариантов является правильным. Задача LLM заключалась в выявлении классов, действительно отраженных в тексте по смыслу, сравнении предложенных вариантов, выявлении ошибок и аргументированной оценке того, какой из них ближе к корректной классификации. Цель данного этапа — получить интерпретируемую оценку расхождений между предсказаниями классификатора, обученного в рамках текущей работы, и эталонной разметкой, а также выявить возможные причины ошибок на уровне смыслового содержания текстов. В качестве LLM использовались RuadaptQwen2.5 QWQ от RefalMachine и YandexGPT 5 Pro от Yandex.

Анализ результатов, полученных от первой LLM (RuadaptQwen 2.5 QWQ), выступавшей в роли аналитика и "судьи" на подвыборке из 100 примеров, позволил выявить характерные закономерности в расхождениях между предсказаниями классификатора и эталонной разметкой. Модель продемонстрировала способность к детальному семантическому анализу, обосновывая выбор меток на основе наличия или отсутствия в тексте явных смысловых указаний на соответствующие классы. В большинстве случаев (90 из 100) модель оценила вариант, соответствующий эталонной разметке, как более корректный, что свидетельствует о значительной доле ошибок классификатора BERT на данной выборке.

Наиболее распространенным типом ошибок, на который указывала модель, было необоснованное добавление лишних классов в предсказания классификатора. Часто это касалось семантически близких классов в области доставки. Модель систематически фиксировала случаи подмены класса 0 ("долгая доставка") на класс 1 ("доставка стала долгой") при отсутствии в тексте признаков изменения ситуации во времени, таких как "последнее время", "стали" (например, в отзыве "Была очень долгая доставка"), и наоборот – пропуск класса 1 при наличии таких маркеров (отзыв "Последнее время доставка занимает значительно больше"). Аналогичная путаница наблюдалась между классами 0 и 3 ("регулярные опоздания"), где разграничение между классами основывалось на наличии или отсутствии указаний на регулярность события (например, ошибочное добавление класса 3 в отзыве "Иногда очень долго везут"; или выбор класса 0 вместо класса 3 в отзыве "Постоянно требуется долго ждать доставки").

К числу выявленных ошибок также относится присвоение нерелевантных классов, в частности, класса 36 ("сравнивают с конкурентами") в контексте нейтральных упоминаний альтернатив (например, "За это время и до магазина самому можно дойти"), или класса 4 ("Не отследить реальное время доставки") при отсутствии соответствующего содержания (например, отзыв "оставляйте номер курьера").

В ряде случаев модель зафиксировала пропуски релевантных меток в предсказаниях классификатора, например, отсутствие класса 15 ("Просроченные товары") при наличии прямого указания на проблему (отзывы: "Часто просрочка", "Привозят просрочку"). В единичных случаях LLM указывала на ограниченность предложенной таксономии или на высокую степень неоднозначности формулировок, затрудняющую однозначную интерпретацию текста (например, "Цены постоянно не постоянные").

Аналогичный эксперимент проведен с использованием второй LLM (YandexGPT 5 Pro), примененной к тем же 100 примерам. Результаты данного анализа подтвердили выводы, полученные на основе первой модели, и в ряде аспектов дополнили их. В 96 из 100 текстов данная LLM признала эталонную разметку более точным.

Как и в первом случае, основным источником ошибок в предсказаниях классификатора, обученного в рамках настоящей работы, является добавление избыточных или нерелевантных классов. Наиболее частыми были ошибки в пределах группы классов, связанных с аспектами доставки, а также в классах, касающихся качества товаров и клиентского сервиса. Модель зафиксировала, что классификатор недостаточно точно различает состояние и его изменение (например, "доставка была долгой" и "доставка стала долгой"), а также в интерпретации частотности событий. Также были указаны систематические случаи игнорирования ключевых лексем, указывающих на повторяемость или единичность действия, таких как "всегда", "иногда", "постоянно".

Таким образом, обе языковые модели продемонстрировали высокую степень согласованности в оценке расхождений между предсказаниями классификатора и эталонной разметкой. Их заключения подтверждают, что основными проблемами классификатора являются склонность к генерации ложноположительных предсказаний, недостаточная чувствительность к семантическим деталям и недостаточная способность к разграничению тематически смежных классов.

# 6 Заключение

В рамках проведенной работы получены следующие основные результаты:

1. Разработана и реализована на языке Python 3 библиотека методов аугментации текстов для работы с русскоязычными наборами данных, которая доступна на платформе PyPI [21].

2. Для демонстрации применимости реализованных методов аугментации выбран подходящий русскоязычный набор текстовых данных и задача классификации.

3. Проведены эксперименты и дана количественная оценка эффективности разработанных методов. Подход, основанный на контекстной замене слов, показал наибольшую эффективность, увеличив значение качества по метрике F1-micro с 78% до 80.3%.

Методы аугментации для русского языка, разработанные в данной работе, могут быть применены под различные задачи обработки естественного языка, в зависимости от специфики используемых данных.

# Список литературы


1. Ilhem Salah, Khaled Jouini, Ouajdi Korbaa. On the use of text augmentation for stance and fake news detection // Information and Telecommunication. — 2023. — April. — Vol. 7, No. 3. — Pp. 359–375.

2. Dai H., et al. AugGPT: Leveraging ChatGPT for Text Data Augmentation // IEEE Transactions on Big Data. – 2025.

3. Markus Bayer, Marc-Andre Kaufhold, Bjorn Buchhold. Data augmentation in natural language processing: a novel text generation approach for long and short text classifiers. // International Journal of Machine Learning and Cybernetics. – 2023. – Vol. 14. – Pp. 135-150.

4. Wei, Jason and Zou, Kai. EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks. // Proceedings of the Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. – 2020. – Pp. 6382-6388.

5. Marzieh Fadaee, Arianna Bisazza, Christof Monz. Data Augmentation for Low-Resource Neural Machine Translation. // Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. – 2017. – Vol. 2. – Pp. 567-573.

6. Sosuke Kobayashi. Contextual Augmentation: Data Augmentation by Words with Paradigmatic Relations. // Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. – 2018. – Vol. 2. – Pp. 452–457.

7. Garg, Siddhant and Ramakrishnan, Goutham. BAE: BERT-based Adversarial Examples for Text Classification. // Proceedings of the Conference on Empirical Methods in Natural Language Processing. – 2020. – Pp. 6174-6181.

8. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. – 2018.

9. Nidhal Baccouri. Deep Translator: a flexible free and unlimited translation library for Python [Электронный ресурс]. – Электрон. дан. – URL: https://github.com/nidhaloff/deep-translator (дата обращения: 10.09.2024).

10. Efficient Estimation of Word Representations in Vector Space / Tomas Mikolov, Kai Chen, G.s Corrado, Jeffrey Dean. – 2013. – 01.

11. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. – 2017.

12. Korobov Mikhail. Morphological Analyzer and Generator for Russian and Ukrainian Languages // CCIS. – 2015. — Vol. 542. – Pp. 320-332.

13. Проект Natasha: набор Python-библиотек для обработки текстов на русском языке [Электронный ресурс]. – 2020. – URL: https://natasha.github.io/ (дата обращения: 10.09.2024).

14. Ilya Segalovich. A fast morphological algorithm with unknown word guessing induced by a dictionary for a web search engine. // MLMTA. – 2003. – Pp. 273-280.

15. Kutuzov A., Kuzmenko E. WebVectors: A Toolkit for Building Web Interfaces for Vector Semantic Models. // CCIS. – 2017. – Vol. 661. – Pp. 155-161.

16. Zmitrovich D., Abramov A., Kalmykov A. A Family of Pretrained Transformer Language Models for Russian // Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024). – 2024. – Pp. 507–524.

17. Yandex Research. RuLeanALBERT — предобученная языковая модель для русского языка [Электронный ресурс]. – Электрон. дан. – URL: https://github.com/yandex-research/RuLeanALBERT (дата обращения: 27.01.2025).

18. Tiedemann J., Thottingal S. OPUS-MT: Building Open Translation Services for the World // Proceedings of the 22nd Annual Conference of the European Association for Machine Translation (EAMT). – 2020. – Pp. 479–480.

19. Набор данных для множественной классификации текстов для автоматизации исследования обратной связи пользователей Самоката – Ecom.tech, Deep Learning School [Электронный ресурс]. – Электрон. дан. – URL: https://ods.ai/competitions/dls_ecomtech (дата обращения: 10.02.2025)

20. Sokolova M., Lapalme G. A systematic analysis of performance measures for classification tasks // Article in Information Processing & Management. – 2009. – Vol. 45, No. 4. – Pp. 427–437.

21. augmenta-ru: библиотека для аугментации русскоязычных текстов [Электронный ресурс]. – Электрон. дан. – URL: https://pypi.org/project/augmenta-ru/ (дата обращения: 27.04.2025).

22. Репозиторий библиотеки augmenta-ru. [Электронный ресурс]. – Электрон. дан. – URL: https://github.com/tymillnyc/augmenta-ru (дата обращения: 27.04.2025).

23. Kuratov Y., Arkhipov M. Adaptation of Deep Bidirectional Multilingual Transformers for Russian Language [Электронный ресурс]. – 2019. – URL: https://www.dialog-21.ru/media/4606/kuratovyplusarkhipovm-025.pdf (дата обращения: 09.09.2024).

24. Tikhomirov M., Chernyshov D. Facilitating Large Language Model Russian Adaptation with Learned Embedding Propagation // Journal of Language and Education. – 2024. – Т. 10. – №. 4. – С. 130-145.

25. Яндекс. YandexGPT 5 — новое поколение нейросетей Яндекса [Электронный ресурс]. – URL: https://ya.ru/ai/gpt (дата обращения: 20.04.2025).

26. Anthropic. Claude 3.7 Sonnet System Card [Электронный ресурс]. – URL: https://www.anthropic.com/claude-3-7-sonnet-system-card (дата обращения: 20.04.2025).

27. Google DeepMind. Gemini 2.5: Our most intelligent AI model [Электронный ресурс]. – URL: https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/ (дата обращения: 20.04.2025).